A Critical Review of MARVEL and MARVEL_GNN

Is a trained graph neural network better than Dijkstra and other deterministic rules?

Internal review (critical position)

15 October 2026

Abstract

This report takes a critical position on purpose. It examines the MARVEL method (Furtenbacher, Császár & Tennyson 2007; Furtenbacher & Császár 2012). It also examines the graph neural network (GNN) that this repository adds to MARVEL. The GNN has one shared encoder and four heads: uncertainty, outlier detection, orphan linkage and label correction. The review used the code at commit `709c4f6` and the working tree. It ran the shipped checkpoint (`data/marvel_gnn.pt`) on the same validation data, seeds and metrics as `gnn/train.py`. The results agree with the project numbers to all printed digits. The review then added simple deterministic rules as baselines. The project did not run these baselines. For three of the four heads, a one-line deterministic rule gives the same result as the GNN or a better result. Each rule uses only data that the GNN already receives as an input feature. For the uncertainty head, the project reports a gain of 2–3 orders of magnitude over legacy MARVEL. When the deterministic rule gets one global scale factor, this gain decreases to a factor of 1.3–1.8 in NLL. The training target of the uncertainty head cannot see the measurement uncertainty of a non-redundant transition. Thus the GNN gives some levels an uncertainty that is smaller than the uncertainty of the only transition to that level. This is physically not possible. MARVEL also has problems. Its "bootstrap" is not a statistical bootstrap. Its Dijkstra uncertainty has no clear statistical meaning. Its outlier rule cannot see errors on bridge transitions.

Contents

1 Scope and method **2**

2 Summary **3**

3 Problems in MARVEL **3**

3.1 The model is linear and Gaussian, thus an exact uncertainty is available 3

3.2 The Dijkstra uncertainty is a loose upper limit 4

3.3 The "bootstrap" is not a statistical bootstrap 4

3.4 The combined uncertainty counts the same uncertainty many times 4

3.5 The outlier rule cannot see leverage 4

3.6 Adjusted uncertainties make the published data consistent 4

3.7 Structural limits 5

3.8 People found the real errors in this project, not models 5

4 Is a GNN necessary? **5**

5 Non-determinism: GNN and Dijkstra **6**

5.1 GNN inference is almost deterministic 6

5.2 The data do not fully determine the trained function 6

5.3 Properties that Dijkstra and WLS guarantee, but the GNN does not 6

6 Uncertainty head **7**
6.1 The target measures inconsistency, not uncertainty 7
6.2 The target of a leaf level does not include its own transition 7
6.3 The comparison with legacy MARVEL is not fair 8
6.4 Other problems in the evaluation 8

7 Outlier-detection head **9**

8 Label-correction head **9**

9 Orphan-linkage head **10**

10 Other design and engineering problems **10**

11 Scientific integrity **11**

12 Evidence that would change the result of this review **11**

A How to reproduce the results **12**

1 Scope and method

Items examined. The review examined these items:

- `src/marvel_gnn/core/`: parser, components, WLS solver and legacy uncertainty.
- `src/marvel_gnn/gnn/`: features, model, corruption, label correction, linkage and training.
- `scripts/eval_so_calibration.py`, `marvel_gnn_plan.md`, `STATUS.md`, `README.md` and the training logs in `data/`.
- The original C++ code (`Source_Code_CPP/MARVEL4.1.cpp`).

Scripts. This directory contains two scripts:

- `skeptic_baselines.py` loads the shipped checkpoint. It makes the validation sets again with `train.prepare`, `train.prepare_corrupted` and `linkage.prepare_linked`. It uses the *same seeds* as `train.py`. It calculates the scores of the GNN and of the deterministic rules on the same samples.
- `seed_run.py` trains the full model five times. Only the `torch` seed changes. All data seeds stay the same. Thus the script measures only the variation that comes from training.

The GNN, legacy and published-e_E results agree with `marvel_gnn_plan.md` (section of 2026-08-24). For example, the GNN NLL is 7.189/6.445 and the legacy NLL is 2123.58/51.71 on `12C17O/13C18O`. Thus the comparison is correct.

Rule for comparison. A claim that "the GNN is better than MARVEL" needs a comparison with the *strongest* simple alternative. A comparison with the weakest alternative is not sufficient. This review supplies the strong alternatives. Each deterministic rule uses only data that the GNN already receives as input features. Some rules also use one global scale factor. The review fits this factor on the *training* isotopologues only.

Head	Metric	GNN	Simple deterministic rule	Rule
Uncertainty	NLL ↓ (`12C17O/13C18O`)	7.19 / 6.44	9.23 / 11.42	WLS covariance × Birge × 1 scale factor
Outlier	AP ↑	0.210 / 0.167	**0.230 / 0.269**	raw $\lvert r\rvert/u$ (input feature 3)
Label correction	top-1 ↑	97.1% / 96.7%	**97.1% / 97.2%**	$\arg\min \lvert E_c - E_l - \nu\rvert/u$
Orphan linkage	MRR ↑	0.128 / 0.052	**0.573 / 0.094**	$\lvert\Delta J\rvert = 1$ first, random order for equal scores

Table 1: The GNN and the simplest correct deterministic rule, on the same validation samples as `train.py`. Bold text shows where the rule is equal to the GNN or better.

2 Summary

1. **Outlier detection.** The project did not compare the GNN with a residual threshold. This review did the comparison. The raw normalized residual gives a higher average precision than the GNN on *both* validation isotopologues. This residual is already the third edge feature of the GNN (§7).
2. **Label correction.** The project calls this head its "strongest head". But one of its input features, used alone, gives the same result (§8).
3. **Orphan linkage.** A ranking by the selection rule $\Delta J = \pm 1$ is better than the GNN. The MRR is 4.5 times higher on `12C17O` and 1.8 times higher on `13C18O` (§9).
4. **Uncertainty.** The project scores the legacy uncertainty against the GNN training target. The legacy uncertainty gets no scale factor. One fitted scale factor removes almost all of the difference. Also, the target does not include the uncertainty of non-redundant transitions. On `13C18O`, the GNN uncertainty of each of the 35 leaf levels is smaller than the uncertainty of its only transition (§6).
5. **Non-determinism.** The GNN inference is almost deterministic. On the GPU, the variation between runs is approximately 10^{-6} in $\log\sigma$. The real problem is different. The trained function changes with the seed, the data split and the stop epoch. Also, the GNN has no audit trail, no guaranteed physical properties and no estimate of its own (epistemic) uncertainty (§5).
6. **MARVEL.** The MARVEL uncertainty method (Dijkstra and "bootstrap") is a heuristic. An exact, fast, deterministic method is available: the WLS covariance, which is the effective resistance of the network. Neither this project nor the C++ code uses it (§3).

3 Problems in MARVEL

3.1 The model is linear and Gaussian, thus an exact uncertainty is available

MARVEL solves a weighted least-squares (WLS) problem. The incidence matrix is B. Each row of B is a transition and each column is a level. The weights are $w_e = 1/u_e^2$:

$$\hat{E} = \arg\min_E \sum_e w_e \left(\nu_e - (E_{u(e)} - E_{l(e)})\right)^2, \qquad L = B^\top W B.$$

The matrix L is a weighted graph Laplacian. MARVEL assumes independent Gaussian errors with standard deviation u_e. With this assumption, the covariance of $\hat{E}_i - \hat{E}_0$ has an exact closed form:

$$\mathrm{Var}(\hat{E}_i - \hat{E}_0) = R_{\mathrm{eff}}(i,0) = (e_i - e_0)^\top L^+ (e_i - e_0).$$

This value is the *effective resistance* between level i and the ground level. The electrical network has a conductance w_e on each edge. The function `gnn/data.py:edge_leverages` already calculates L^+ for the leverage feature. Thus the project calculates the exact uncertainty but does not use it.

3.2 The Dijkstra uncertainty is a loose upper limit

Legacy MARVEL gives $\sigma_i^{\mathrm{SP}} = \sqrt{\min_{\mathrm{paths}} \sum u_e^2}$. This value is the resistance of the single best path. Rayleigh monotonicity gives $R_{\mathrm{eff}} \leq R_{\text{shortest path}}$. The two values are equal only when the network is a tree. Thus the Dijkstra method has these problems:

- **It ignores redundancy.** Parallel paths decrease the true variance. They do not decrease the Dijkstra value. The best-determined levels get the largest overestimate.
- **It ignores inconsistency.** The method uses only u_e. It does not use residuals. A level with contradictory transitions gets the same σ as a level with fully consistent transitions.
- **It depends on the root.** The result changes when the reference level changes. For a floating component, the reference level is arbitrary.

Thus the question "why a GNN and not Dijkstra?" starts from an incorrect premise. The correct deterministic comparison is not Dijkstra. It is R_{eff}, with a goodness-of-fit scale factor.

3.3 The "bootstrap" is not a statistical bootstrap

The code at `MARVEL4.1.cpp:1095--1110` changes each uncertainty to $u_e \leftarrow k\, u_e^{\mathrm{orig}}$, with $k \sim \mathcal{U}\{1, \ldots, 10\}$. Then it solves again. It gives $\sqrt{2}\max(|E - \tilde{E}|, s)$ as the result.

- It does not resample the *data*. It changes only the *weights*. If the data are fully consistent, each new solution gives the same energies. Then the "bootstrap" uncertainty is zero, for all values of the measurement uncertainties. The method measures the sensitivity to the weights. This is a sign of inconsistency, not a sampling distribution.
- There is no derivation for the range 1..10, the factor $\sqrt{2}$ or the maximum of the two values.
- **The results are not reproducible.** The code initializes the random generator from `std::random_device` (`mt19937 gen(rd())`). There is no option to set the seed. Thus legacy MARVEL is *also* non-deterministic. If non-determinism is a problem for the GNN, it is also a problem for the published MARVEL uncertainties.

3.4 The combined uncertainty counts the same uncertainty many times

The function `combined_unc` runs a second Dijkstra search. In this search, the cost to *enter* a level is $\max(\sigma^{\mathrm{SP}}, \sigma^{\mathrm{boot}})^2$. Each level cost already includes the uncertainty of the path to that level. Thus the sum along a path adds the same upstream uncertainty many times. The result has no statistical meaning. It increases approximately with the square of the path length.

3.5 The outlier rule cannot see leverage

MARVEL puts each transition into one of these groups: WRONG, VERY BAD, VERY BAD_100 or VERY BAD_1000. It uses a fixed ratio $|r_e|/u_e$ (`MARVEL4.1.cpp:64--90, 730`). But the fit absorbs a fraction h_e of each error. This fraction is the leverage. Thus the residual is $(1 - h_e)$ times the true error. A bridge has $h_e = 1$, so its residual is *always* zero. The project measured that 44–52% of the transitions in the small validation networks are bridges. Standard regression diagnostics are available and deterministic. Examples are the studentized deleted residual, Cook's distance and a "cannot verify" flag for $h \approx 1$. MARVEL uses none of them. In the usual MARVEL workflow, a person increases the uncertainties until the flags stop. This can hide an error behind other errors (masking). It can also flag correct transitions (swamping).

3.6 Adjusted uncertainties make the published data consistent

Published MARVEL input files contain *adjusted* uncertainties. A person increases the uncertainties until the network is consistent. Thus a method that trains on the residuals of these data learns from data that a person already made consistent. This is a circular process. Also, the deposited CO

data are not all consistent. This review calculated the Birge ratio $\sqrt{\chi^2/\nu}$ of each main component: 0.72 (`13C16O`), 0.81 (`12C18O`), 0.28 (`13C18O`), 4.9 (`12C17O`) and **30.9** (`13C17O`). For `13C17O`, the residuals are 31 times the stated uncertainties. Thus this training isotopologue is not "clean" in a statistical sense. This component also sets the uncertainty scale of the GNN (§6).

3.7 Structural limits

- **The label is the identity.** The label string identifies the level. An incorrect label does not cause an inconsistency. It makes a different node. This node is frequently a new leaf with a zero residual. In regions with resonance mixing (for example the CO_2 Fermi index r), a label is not physically unique.
- **Floating components.** The default is `--minsize 200`. With this value, MARVEL removes small components and gives no message. With a small `minsize`, each component gets its own arbitrary zero. MARVEL cannot connect these zeros.
- **Selection-rule limits.** In SO, the e/f parity split (2026-09-14) is a selection-rule limit. No electric-dipole transition can connect the two parities. Thus no method, classical or learned, can connect these two networks with transition data.
- **Systematic errors.** In real spectroscopy, the largest errors usually come from each source as a whole. Examples are calibration offsets, pressure shifts and unit errors. These errors are not independent. Thus WLS with one weight for each transition gives uncertainties that are too small.

3.8 People found the real errors in this project, not models

Each real data error in this project was found by residual calculations, physical checks or code review. A learned model found none of them. The real errors were:

- SO sources in MHz with the label cm^{-1} (2026-09-14).
- An incorrect F-label formula for 22HeStLy (2026-09-18).
- MHz data in the exported `MARVEL_trans.txt` files.
- Exchanged CO_2 deposit files.
- The uninitialized variable `hasSegmentfile` in the C++ code.

These failure types are the important types in practice. The synthetic corruption model of the GNN contains none of them (§7).

4 Is a GNN necessary?

The forward problem is linear and the estimator is exact. A model can learn only the parts where the WLS assumptions are wrong. These parts are: incorrect uncertainties, incorrect transitions and incorrect labels. For each part, a known deterministic method is available. Each method is easy to interpret.

Task	Deterministic method	Does the GNN add value?
Uncertainty	L^+ covariance; Birge or per-source scale factors; hierarchical variance model	not shown (§6)
Outliers	Studentized residual, leverage, robust (Huber/IRLS) fit	no (§7)
Label correction	Combination-difference test on the candidate labels	no (§8)
Orphan linkage	Effective-Hamiltonian or Dunham prediction of absolute energies	no (§9)

A GNN can approximate many functions of local graph neighbourhoods. But here, the correct function is either known in closed form (WLS covariance, residual diagnostics), or it needs data that the GNN cannot get (absolute energies across components). Thus the GNN must show that it adds value. The data in this report show that it does not.

5 Non-determinism: GNN and Dijkstra

A common objection is: "a GNN is non-deterministic, but Dijkstra is deterministic." This objection needs a precise statement. In its simple form, it is easy to reject.

5.1 GNN inference is almost deterministic

On the CPU, two forward passes of the shipped checkpoint give identical results. On the GPU (RTX 2070, with the CUDA build of the project), the review did 20 forward passes on `13C16O`. The maximum difference was $\max|\Delta \log \sigma| = 1.9 \times 10^{-6}$. This difference comes from atomic scatter-add operations. The difference between GPU and CPU was 2.4×10^{-6}. These values give a relative change in σ of approximately 10^{-6}. This change is not important. Also, the legacy MARVEL bootstrap is less reproducible than this.

5.2 The data do not fully determine the trained function

The real problem is this: σ_i depends on the training run, not only on the data.

- **Seed.** Table 2 shows five training runs. The data, the split and the corruption seeds are the same in each run. Only the seed for the initial weights changes.
- **Data split.** The project moved one isotopologue between the training set and the validation set (`13C17O`↔`13C18O`). `12C17O` was in the validation set in *both* runs. Its label-correction accuracy changed from 82.9% to 97.1%. The ratio between the legacy NLL and the GNN NLL changed from approximately 600 to approximately 8. Thus the results describe the split as much as they describe the method.
- **Stop epoch.** The validation NLL decreases to 90 at epoch 300. Then it increases to 265 at epoch 950 (`data/train_v6.log`). Thus the model overfits strongly. The project selects the checkpoint with the *same* isotopologues that it then reports as validation results. Thus there is no independent test set.
- **Version changes.** Each new training run changes all reported uncertainties. MARVEL data go into reference databases such as ExoMol and HITRAN. To reproduce one σ, a database must keep the weights, code, seeds and hardware details for each version.

(seed results pending)

Table 2: Seed variance.

5.3 Properties that Dijkstra and WLS guarantee, but the GNN does not

1. **Audit trail.** Dijkstra can explain a value of σ with a path. L^+ can explain it with a sum of contributions. The GNN gives only a number from three attention layers.
2. **Monotonicity.** In WLS, a new measurement cannot increase the formal variance of a level (Rayleigh). The GNN does not enforce this property. If a person adds a transition, the GNN σ can increase.
3. **Lower limit.** If one transition connects a level, then $\sigma \geq u_e$. The GNN does not obey this limit (§6.2).

4. **Scale covariance.** If all u_e are multiplied by c, each deterministic σ is also multiplied by c. The GNN receives $\log_{10} u/10$ through fixed normalizations and a trained bias. On SO, the GNN gives an almost constant $\sigma \approx 5 \times 10^{-4}$ cm^{-1} for all networks (note of 2026-09-17).

5. **Epistemic uncertainty.** The project uses no ensemble and no Bayesian method. Thus the GNN gives σ without an uncertainty on σ. This is also true outside the training distribution (SO, CO_2), where the error is largest.

Result. "Non-determinism" is not the correct word for the problem. Also, legacy MARVEL is non-deterministic. The real problem is that the data do not fully determine the GNN output: the seed, the split and the epoch all change it. Also, the GNN has no audit trail. It does not obey properties that the deterministic estimators obey by design.

6 Uncertainty head

6.1 The target measures inconsistency, not uncertainty

The training target comes from `data.py:refit_error_matrix`. The procedure removes 15% of the transitions and solves again. Then it records $E_i^{\text{masked}} - E_i^{\text{full}}$. This procedure has three structural problems:

1. **Consistent data give a zero target.** If all transitions agree exactly with the solution, each new solution gives the same energies. Then the target σ is zero, for all values of u_e. Thus the head learns how inconsistent the data are. It does not learn how well the data determine each level.

2. **The scale is not defined.** The procedure is a delete-d jackknife without the $(n-d)/d$ correction. For a correct model and a redundant network, $\sigma_{\text{shift}} \approx \sqrt{d/(n-d)}\,\sigma \approx 0.42\,\sigma$. The plan says that the target is "sensitivity under 15% removal, not an absolute error scale". But the project then compares the head with the published e_E, which has an absolute scale.

3. **Censoring.** Sometimes the procedure removes the only transition to a level. Then the level is not in the ground component, and its value is NaN. Thus the loss does not include the samples where the level is least certain.

6.2 The target of a leaf level does not include its own transition

Let ℓ be a level with only one transition e, to parent level p. When e stays in the sample, the solution is $\hat{E}_\ell = \hat{E}_p + \nu_e$. Thus the target for ℓ is *identical* to the target for p. The value u_e is not in the target. The physically correct uncertainty is $\sqrt{\sigma_p^2 + u_e^2}$. The table shows the values for the validation data:

	`12C17O`	`13C18O`
levels / leaf levels	33 / 4	345 / 35
max $\lvert\text{target}_\ell - \text{target}_p\rvert$ (μcm^{-1})	1.9×10^{-6}	6.1×10^{-5}
median $\sigma_\ell^{\text{GNN}}/\sigma_p^{\text{GNN}}$	0.998	1.001
median $u_e/\sigma_\ell^{\text{GNN}}$	0.18	**4.54**
leaves with $\sigma_\ell^{\text{GNN}} < u_e$	0/4	**35/35**

The target differences are only float32 noise. The GNN learned $\sigma_\ell = \sigma_p$ exactly. On `13C18O`, the GNN uncertainty of *each* leaf is smaller than the stated uncertainty of the one transition that determines that leaf. The median factor is 4.5. More training cannot correct this problem, because the loss function rewards it.

6.3 The comparison with legacy MARVEL is not fair

The project scores the legacy σ and the published e_E with NLL on the GNN target distribution. This distribution is the masked-refit shifts on the validation isotopologues. The legacy σ is a full-propagation uncertainty, with a very different scale. It gets no scale factor. But the GNN learns the scale. Table 3 adds deterministic uncertainty shapes. Each shape gets *one* global scale factor. The review fits this factor by closed-form maximum likelihood on the three training isotopologues.

	`12C17O` (33 levels)		`13C18O` (345 levels)	
Source of σ	NLL	1σ cov.	NLL	1σ cov.
GNN (shipped checkpoint)	**7.19**	93.3%	**6.44**	97.8%
legacy Dijkstra + bootstrap (project value)	2123.58	85.8%	51.71	99.4%
published e_E	8.33	96.1%	13.06	99.6%
Dijkstra only, no scale factor	2864.34	83.7%	179.48	99.3%
exact WLS $\sqrt{R_{\text{eff}}}$, no scale factor	2882.05	82.8%	186.01	99.3%
exact WLS × Birge ratio	124.18	87.5%	2247.36	98.5%
Dijkstra × 1 fitted scale factor	11.11	100%	16.47	100%
exact WLS × 1 fitted scale factor	10.94	100%	15.78	100%
exact WLS × Birge × 1 fitted scale factor	9.23	99.6%	11.42	99.7%

Table 3: Uncertainty NLL (a lower value is better; σ in μcm^{-1}) on the validation masked-refit errors from `train.py`. The fitted scale factors are 2782 (Dijkstra), 2790 (R_{eff}) and 101 (R_{eff}×Birge).

Interpretation.

- One parameter decreases the Dijkstra NLL from 2864 to 11.1. Thus the gain of "2–3 orders of magnitude" comes almost fully from the scale. The project compares a value with no scale factor to a value with a fitted scale.
- The real GNN gain over the best one-parameter rule is $\Delta\text{NLL} \approx 2$ on `12C17O` and ≈ 5 on `13C18O`. This gain is real but small. The review measured it on the GNN target, on two isotopologues. One of these isotopologues has only 33 levels.
- The fitted scale factor is approximately 2800 times the formal WLS σ. With the Birge correction, it is approximately 100 times. A single maximum-likelihood scale also gives approximately 100% coverage at 1σ. These two results show that the target has heavy tails and many zeros. Most masks do not move a level. A small number of masks move the level through inconsistent transitions. A Gaussian NLL is not the correct likelihood for this distribution. Thus *each* method, also the GNN, gives too much coverage at 1σ (93–98%, but the target is 68%).
- The Spearman correlation between the GNN σ and the exact WLS σ is -0.14 on `12C17O` and 0.62 on `13C18O`. On the small network, the GNN ranking has no relation to the formally correct ranking.

6.4 Other problems in the evaluation

- **A check that is always true.** The check "$|E_{\text{ours}} - E_{\text{pub}}| \leq \sigma$: 100%" is always true. The port gives the published energies to 10^{-8} cm^{-1}. Thus the left side is almost zero.
- **A small, correlated validation set.** The five CO isotopologues have almost the same topology and QN structure. Thus "generalization across isotopologues" is interpolation. `12C17O` has only 33 levels and 50 transitions.
- **No uncertainty on the metrics.** Each result comes from one run, one split and one bootstrap seed. There are no confidence intervals.

Result. The head optimizes a target with no absolute scale. This target does not include the measurement uncertainty of non-redundant transitions. Most of the reported gain over legacy MARVEL comes from the scale. A one-parameter deterministic rule gives almost the same NLL. A deterministic rule never gives $\sigma_\ell < u_e$.

7 Outlier-detection head

The plan does not compare the GNN with the residual-ratio rule. The plan says that this comparison "risks inheriting its blind spots". But a comparison does not give blind spots to a model. A comparison only shows if the model adds value. The review did the comparison on the same 50 corrupted copies of each validation isotopologue:

	`12C17O` (base rate 3.9%)			`13C18O` (base rate 4.9%)		
Score	AP	AP freq	AP QN	AP	AP freq	AP QN
GNN	0.210	0.041	0.295	0.167	0.027	0.211
raw $\log(1+\lvert r\rvert/u)$ (edge feature 3)	**0.230**	**0.044**	**0.328**	**0.269**	0.027	**0.401**
studentized residual (edge feature 5)	0.219	0.042	0.317	0.249	0.026	0.366

Table 4: Average precision on the corrupted validation copies. One input feature is better than the full GNN on both isotopologues. For QN changes on `13C18O`, it is 1.9 times better.

Other problems.

- **Training data and test data come from the same synthetic generator.** The generator corrupts 5% of the transitions. Each corruption is a log-uniform shift of 5–500u or a $J \pm 1$ change on the upper level. Thus the test measures how well the GNN inverts its own generator. It does not measure how well the GNN finds real errors.
- **The real failure types are not in the generator.** The real types are: unit errors (a full source scaled by 1/29979); systematic label errors in a full source (the 22HeStLy F-label error affected approximately 60% of a network, not 5%); exchanged deposit files; air and vacuum wavenumbers; exchanged upper and lower levels; incorrect e/f and K_a/K_c labels; and transitions from the incorrect isotopologue. Each real error in this project was of one of these types.
- **The "structural floor" is a deterministic fact.** No method can find an error on a bridge ($h = 1$). A deterministic output can show h for each transition. This needs one line of code and no model.

Result. The GNN is worse than one of its own input features. The decision to not compare the GNN with the residual rule hid this result.

8 Label-correction head

The candidate window contains the levels with the same non-J tokens and with J within ± 2. The corruption is exactly a $J \pm 1$ change on the upper level. The evaluation uses only the "recoverable subset", where the true upper level is in the window. The candidate feature `extra[2]` $= \log(1 + |E_c - E_l - \nu|/u)$ is the classical combination-difference test. The plan already calls it "the dominant feature".

	12C17O ($n = 35$, window 3.3)		13C18O ($n = 720$, window 3.9)	
Ranking method	top-1	MRR	top-1	MRR
GNN	97.1%	0.979	96.7%	0.982
arg min mismatch (one feature, no model)	**97.1%**	**0.986**	**97.2%**	**0.986**

- The "random 30%" baseline is too weak to be useful. The project designed the task so that one arithmetic check solves it.
- The task is narrow by design. It has one QN, one direction (upper level), one error type (± 1) and only the recoverable cases. Real label errors include $\Delta J = \pm 2$, parity, K_a/K_c exchanges, vibrational label errors in polyads and errors on the lower level.
- The selection has two stages. First, the outlier head must flag the transition ($p > 0.5$). Then the label-correction head runs. Table 1 shows that the outlier head is the weak stage.

Result. The head gives the same result as a deterministic combination-difference test. It adds no measurable value. It can be a useful tool, but it is not evidence for the GNN.

9 Orphan-linkage head

	12C17O (109 pairs)			13C18O (884 pairs)		
Ranking method	MRR	hits@1	hits@5	MRR	hits@1	hits@5
GNN	0.128	0%	20%	0.052	0%	12.2%
$\lvert\Delta J\rvert = 1$ first, random order for equal scores	**0.573**	**33%**	**100%**	**0.094**	**2.7%**	**12.8%**

- **The GNN is worse than the selection rule that it receives as a feature.** The $|\Delta$QN$|$ block is an input to the head. But a rule that uses only $|\Delta J|$ has an MRR 4.5 times higher than the GNN on 12C17O.
- **The task is not well posed.** The scientific result is not *which* bridge transition the procedure removed. Without a measured frequency, each pair with $\Delta J = \pm 1$ agrees equally well with the data. The necessary result is the *absolute energy offset* of the orphan component. An effective-Hamiltonian or Dunham fit to the main component gives this offset directly. For a diatomic molecule, the accuracy is frequently 10^{-3} cm^{-1} or better. By design, the GNN never sees absolute energy across components.
- **The training molecule has no real orphan case.** The CO networks have no real orphan components. The project found one real orphan case (SO). That case is a parity limit. No linkage model can or must connect it.

Result. Remove this head, or replace it with an effective-Hamiltonian prediction of the offset. The project documentation already calls the head weak. The data show that it is worse than a one-line rule.

10 Other design and engineering problems

- **Multi-task coupling.** The loss weights of 10× have no justification. Early stopping uses only the uncertainty NLL. Thus the other three heads use the weights from the best uncertainty epoch, which can be a bad epoch for them.
- **Features.** The QN tokens are divided by 50. The code changes e/f to 0/1. Energies go in as $\log(1 + E)/10$. The code does not support categorical tokens (electronic state, F1/F2/F3). For this reason, the _encode_so patch is necessary at evaluation time. The project did not test the "schema-oblivious" claim on asymmetric tops.

- **Scalability.** `edge_leverages` inverts an $n \times n$ matrix. This stops CO_2 training. The deterministic baselines need the same diagonal of L^+. A sparse Cholesky factorization with selected inversion (Takahashi recursion) calculates it exactly at CO_2 size. When this calculation is available, the deterministic baseline is also available.
- **SO evaluation outside the training distribution.** There is no ground truth. The "badness" value (max $|r|/u$) comes from the same solution. The negative correlation of the legacy method has no explanation. The project already withdrew one earlier version of this result. A Spearman ρ over hundreds of dependent levels, without confidence intervals, is weak evidence.
- **Documents that are not current.** `README.md` gives a label-correction accuracy of 88.6%/83.6%. These values come from an old split. `STATUS.md` (2026-07-29) says that the C++ code is not built. The changes to the split and the path in `train.py` are not committed. The file `train_v7_13C18O.log` uses UTF-16, so standard tools cannot read it.

11 Scientific integrity

The project rule "advisory only" for linkage and label correction is correct. But these two heads are the least important, because deterministic rules give the same results. The uncertainty head is different. Its output replaces the published σ, so it can go into a database without human review. This head has the incorrect target (§6.2). Uncertainties go into line lists (ExoMol, HITRAN). From there, they go into calculated abundances. A σ that is smaller than the uncertainty of its own measurement is a scientific-integrity problem. This is the type of problem that `CLAUDE.md` of this project describes.

12 Evidence that would change the result of this review

1. **Use the correct deterministic baseline.** Replace "legacy Dijkstra+bootstrap" with the exact WLS covariance. Add a Birge factor or a per-source variance model (a hierarchical model with one scale factor for each source, fitted by REML). Claim a GNN gain only when the GNN is better than this baseline.
2. **Correct the target.** Measure the calibration on *validation transitions*. Predict ν_e from $\hat{E}_u - \hat{E}_l$. Test the z-score against $\sqrt{\sigma_u^2 + \sigma_l^2 - 2\,\mathrm{cov} + u_e^2}$. This test works at the measurement level. It has an absolute scale, and it includes leaf transitions.
3. **Enforce physics.** Use $\sigma_\ell \geq u_e$ as a hard lower limit. A better method is to predict a *correction factor* on the WLS covariance. Then the model obeys the properties by design.
4. **Compare each head with its rule**, as in Table 1. Report the GNN only where it is better.
5. **Use real errors.** Different MARVEL releases of the same molecule (for example the updates for $H_2{}^{16}O$) contain transitions that a person removed, weighted again or labelled again. These are *real* errors with human labels. They are a much better test set for outlier detection and label correction than synthetic $J \pm 1$ changes. The SO errors of this project (units, F labels) are also real cases.
6. **Use correct statistical procedures.** Keep a test set that early stopping does not use. Use k-fold validation over isotopologues. Use 5 seeds or more and give the mean $\pm$ standard deviation. Use deep ensembles to estimate the epistemic uncertainty.
7. **Orphans.** Compare the head with an effective-Hamiltonian prediction of the offset (Dunham, PGOPHER, Duo). Or remove the head.

Where a learned model can possibly add value

A learned model can possibly add value in these areas, where deterministic methods are difficult: (i) learn the *per-source* systematic-error scale from many molecules, as a prior that replaces manual adjustment; (ii) rank suspect transitions *for human review*, where only the order is important and the rule-based scores above are the input features; (iii) transfer priors between molecules that use the same experimental method. The current heads do none of these tasks. The project has not shown any of them.

A How to reproduce the results

Use a Python 3.12 environment with `torch`, `torch_geometric`, `numpy`, `scipy` and `networkx`. A CPU build is sufficient. Run these commands from the repository root:

```
python -W ignore reports/2026-10-15_skeptic_review/skeptic_baselines.py
python -W ignore reports/2026-10-15_skeptic_review/seed_run.py <seed>   # 0..4
```

Both scripts use the fixed WSL path `/mnt/c/Code/MARVEL_scraping/molecules/CO` for the CO data. `seed_run.py` writes its checkpoint into its own directory. It does not write over `data/marvel_gnn.pt`. The raw results are in `skeptic_results.json` and `seed*.log`.